

---

# ECE661 Fall 2024: Homework 2

TA: Rahul Deshmukh (deshmuk5@purdue.edu)

Due Date: Midnight, 04 Sep 2024

Late submissions will be accepted with penalty: -10 points per-late-day, up to 5 days.

---

In this homework, you will estimate homographies between a given set of images and use the estimated homographies to transform images. Note that you can NOT use any built-in functions to compute homography matrices or for image warping.

Turn in typed solutions via BrightSpace. Additional instructions can be found at BrightSpace.

## 1 Programming Tasks

You will be calculating a homography between two images by manually recording the pixel coordinates of a set of corresponding points in two images and using these coordinates to calculate the unknown elements of the homography that relates the images.

In what follows, Section 2 gives an overview of the problem definition. Section 3 has a list of recommended libraries and tools. The two programming tasks for this homework are defined in Section 4 and Section 5. Finally, the homework submission instructions are given in Section 6.

## 2 Overview

The goal of this homework is to apply your conceptual understanding about homography estimation to transform one image onto another. For homography estimation, we need corresponding points between an image pair. For this homework, you will find the coordinates for the corresponding points using one or more of the recommended tools given in the following section. Then show your results for the tasks given in Section 4 and Section 5.

## 3 Recommended Tools and Libraries

We provide tips and instructions here for linux users. Windows users can install WSL and then follow the given instructions.

## Programming Language

Python with scikit-image or OpenCV. It is highly recommended to learn how to manage environments as opposed to installing libraries system-wide. Therefore, we want you to develop your code in an Anaconda [1] environment (“conda” for short) in order to get you ready for future homework assignments that would be easier to execute in such an environment.

After you have implemented your solutions in your conda environment, we ask you to submit the environment.yml file along with your .py files. You can generate the .yml file as mentioned below.

## Setting up your Anaconda environment

A very useful cheatsheet on the conda commands can be found at [2]. Below is an abbreviated list of commands to get you started:

- If you are used to using pip, execute the following to download Anaconda:

```
sudo pip install conda
```

For alternatives to pip, follow the instructions here [3] for installation.

- Create your ECE661 conda environment:

```
conda create --name ece661 python=3.8
```

- Activate your new conda environment:

```
conda activate ece661
```

- Install the necessary packages (e.g. Scikit-Image, NumPy) for your solutions

```
conda install numpy scikit-image -c conda-forge
```

- After you have created the conda environment and installed the necessary packages, use the following command to export a snapshot of the package dependencies in your current conda environment:

```
conda env export > environment.yml
```

## Tools to record annotated pixel coordinates from an image

Here are a few recommended tools for recording your annotated pixel coordinates from an image:

- GUI-based tools: GIMP [4] for linux users and IrfanView [5] or GIMP for Windows users.
- Command-line tools: ImageMagick [6] has a nice collection of command-line image manipulation tools. You can also use this for resizing your images.

- You can use Matplotlib to display your image and hover your mouse pointer over the figure window. You will see the point coordinates in the status bar.
- You can use the python based tool described in [7]. This method will probably require the **least amount of manual work**.

In your report, let us know which method worked the best for you.

## 4 Task-1

You are given four images which are shown in Figs. 1(a), 1(b), 1(c), and 1(d). The first three images consist of different views of a wall-mounted photo-frame. The fourth image is of **Alex Honnold** climbing El Captain without a rope or safety equipment, becoming the first person to free solo the route.

Complete the following tasks using these images.

1. Using just four points, pick a region of interest (ROI) from the image in Fig. 1(d) and project that region on the PQRS frames shown in Figs. 1(a), 1(b), and 1(c). For this task, you need to find the homographies between the following pairs of images: (1) images shown in Figs. 1(d) and 1(a), (2) images shown in Figs. 1(d) and 1(b), and (3) images shown in Figs. 1(d) and 1(c).
2. Find homographies between images shown in Figs. 1(a) and 1(b), and between images shown in Figs. 1(b) and 1(c). Then apply the product of the two homographies to the image shown in Fig. 1(a). The resulting image should look similar to the image shown in Fig. 1(c).
3. Using only affine homographies, map Alex's image to the PQRS regions in all photo-frame images. Report the results. If one of the results is better than the others, explain why.

You can use any tool recommended in Section 3 to determine the pixel coordinates of a point in an image.

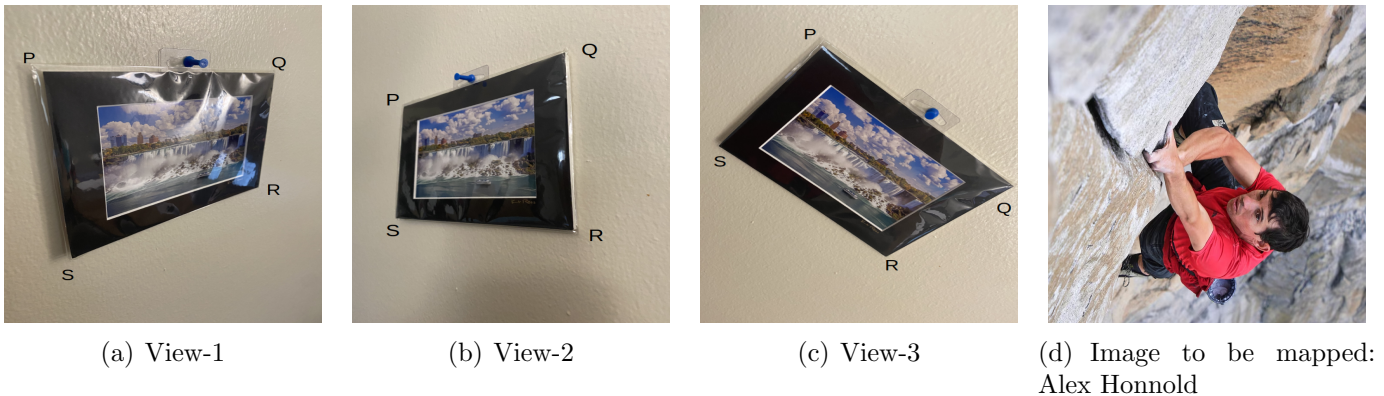


Figure 1: Images for Task1. Note that both outer or inner corner points of the given photo-frame images are acceptable solutions. Download the input images provided for Task1 separately.

## 5 Task-2

Repeat the steps of Task 1 using your own images. You can capture three images of a planar surface from three different viewpoints such as the ones shown in Figs. 1(a), 1(b), and 1(c). For the fourth image you can use your own picture or obtain a picture of an animal, celebrity, etc.

## 6 Submission Instructions

Include a typed report explaining how you solved the programming tasks.

1. Turn in a zipped file with a typed pdf report with source code files. Include your input and output images in the report itself. Rename your .zip file as hw2\_<First Name><Last Name>.zip and follow the same file naming convention for your pdf report too.
2. Your pdf must include a description of:
  - The logic that you used to solve the given tasks.
  - The steps that you used to compute the homographies including equations.
  - Your own input images for Task 2 and output images for the two tasks.
  - Your source code. Make sure that your source code files are adequately commented and cleaned up.
3. Indicate the points that you used on each image to obtain the homographies.
4. In order to avoid large file size of your submission, include JPEG images in your report for showing your results and your input images for Task2.
5. The sample solutions from previous years are for reference only, it's important not to get too biased by those solutions. You're free to format your report however you like as long as it addresses all the required components in the given programming tasks. There are many possible ways to format your report. For example, you're free to generate your pdf report using Jupyter notebook with inline explanation and results, and include your python code (.py files) separately. **Your code and final report must be your own work.**

## 7 Trivia

If you were impressed by [Alex Honnold's wiki page](#), then you should definitely watch Nat-Geo's documentary called "Free Solo" which features his El Captain ascent. Watching the documentary and completing the ECE661 homework is one of the best ways to spend Labor Day long weekend. :)

## 8 Extra Credit (+20 Pts)

In Lecture 3, we discussed the hierarchy of transformations for homographies and how different homography transformations change the lines and shapes in an image. These concepts are directly applicable to image-editing applications (e.g. **iPhone Photos** and **Snapseed by Google**). These applications are based on the concept of foreground and background, and the editing tools allow you to manipulate the foreground objects in relation to the background. You are also allowed to apply global transformations to an image. For example you could just rotate the entire image. When the foreground objects are manipulated, it is done by applying a homography to the whole image.

Specifically, the **iPhone Photos** application offers the following three operations – (1) **Rotating the image**; (2) **Vertically tilting the image**; and (3) **Horizontally tilting the image**. For Android users, the Snapseed tool does similar things. Additionally, you can visit this [web-post](#) which provides example images for the three operations.

The homographies that are used to produce the effects mentioned above can be parameterized by a single parameter, which represents the degree of change required (e.g. the rotation angle).

Your task for the extra credit is the following:

1. **Formulate the parameterized homographies** for each of the three types of operations mentioned above. Representing the parameter by  $\alpha$ , provide the formula for the corresponding 3x3 homography  $H(\alpha)$ .
2. **Apply the homographies** to an input image with horizontal and vertical grid lines and display the resulting images. Your figures should resemble the examples shown in Fig. 2. Present multiple examples (more than one) for each type of editing operation, along with the corresponding  $\alpha$  values.
3. **Analyze the effects** of vertical and horizontal tilting operation on arbitrary vertical and horizontal lines. Discuss which types of lines (vertical or horizontal) remain invariant under each operation. Provide either a proof or numerical evidence to support your conclusions.

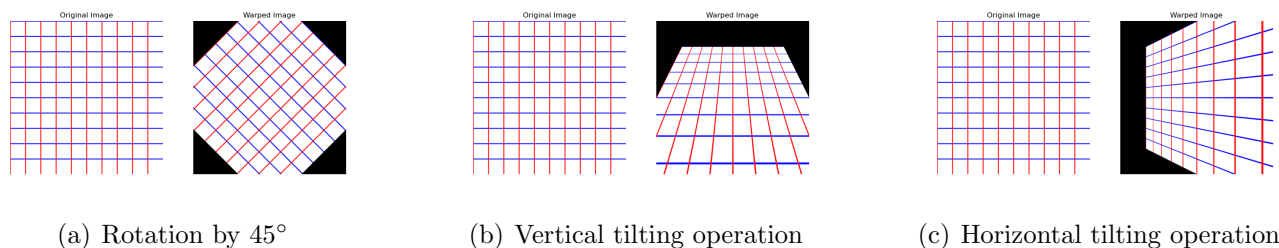


Figure 2: In each sub-plot, the left side shows the input image with grid lines, and the right side displays the output image produced by each editing operation.

**NOTE:** Only for the extra credit, you are allowed to use the `cv2.warpPerspective()` function for creating your output image

## Hints for Extra Credit

The three types of editing operations shown in Fig. 2 are performed with the image center positioned at the origin. To achieve this, you will need to construct both a “normalizing” and a “de-normalizing” homography. These homographies will scale the image coordinates from  $\{(x, y) \in \mathbb{R}^2 | x \in [0, W], y \in [0, H]\}$  to  $\{(u, v) \in \mathbb{R}^2 | u \in [-1, 1], v \in [-1, 1]\}$ . The two homographies are given by Eq. 1.

$$\mathbf{H}_{norm} = \begin{bmatrix} 2/W & 0 & -1 \\ 0 & 2/H & -1 \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{H}_{de-norm} = \begin{bmatrix} W/2 & 0 & W/2 \\ 0 & H/2 & H/2 \\ 0 & 0 & 1 \end{bmatrix} \quad (1)$$

Here  $H, W$  is the height and width of the input image.

Once you have formulated your parameterized homography,  $\mathbf{H}(\alpha)$ , you will need to compute the composite homography  $\mathbf{H}_c(\alpha) = \mathbf{H}_{de-norm} \mathbf{H}(\alpha) \mathbf{H}_{norm}$ . You can then use the `cv2.warpPerspective()` function along with the composite homography to create your output image.

Formulating the rotational homography should be straightforward. For the vertical and horizontal tilting operations, use the fact that either vertical or horizontal lines do not change direction under the specific operation. As discussed in Lecture 3, you also know that lines transform by  $\mathbf{l}' = \mathbf{H}^{-T} \mathbf{l}$ .

Finally, you can use the example shown in Listing 1 as guidance for providing numerical proof for Task 3 of the extra credit.

## Takeaway

If you completed the extra-credit tasks, you have successfully designed the three image editing operations. In the iPhone Photos application, the user provides a value for the scalar parameter  $\alpha$  (using a slider button) and the application displays the output produced by the correction homography.

Listing 1: Example of numerical proof for Task 3 of extra credit

```
The input horizontal lines and vertical lines are:
The horizontal lines are:
[[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 1.  1.  1.  1.  1.  1.  1.  1.  1.  1.]
 [-5. -4. -3. -2. -1.  0.  1.  2.  3.  4.]]
The vertical lines are:
[[ 1.  1.  1.  1.  1.  1.  1.  1.  1.  1.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [-5. -4. -3. -2. -1.  0.  1.  2.  3.  4.]]
-----
Vertical Tilting
horizontal lines transform to
```

```

[[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0. ]
 [-1.5 -1. -0.5 0.  0.5 1.  1.5 2.  2.5 3. ]
 [-5.  -4.  -3.  -2.  -1.  0.  1.  2.  3.  4. ]]
vertical lines transform to
[[ 1.  1.  1.  1.  1.  1.  1.  1.  1.  1. ]
 [-2.5 -2. -1.5 -1. -0.5 0.  0.5 1.  1.5 2. ]
 [-5.  -4.  -3.  -2.  -1.  0.  1.  2.  3.  4. ]]
-----
Horizontal Tilting
horizontal lines transform to
[[-2.5 -2. -1.5 -1. -0.5 0.  0.5 1.  1.5 2. ]
 [ 1.  1.  1.  1.  1.  1.  1.  1.  1.  1. ]
 [-5.  -4.  -3.  -2.  -1.  0.  1.  2.  3.  4. ]]
vertical lines transform to
[[-1.5 -1. -0.5 0.  0.5 1.  1.5 2.  2.5 3. ]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0. ]
 [-5.  -4.  -3.  -2.  -1.  0.  1.  2.  3.  4. ]]

```

## References

- [1] “Anaconda - Managing Environments.” <https://docs.conda.io/projects/conda/en/latest/user-guide/getting-started.html>. 2
- [2] “Conda Cheat Sheet.” [https://docs.conda.io/projects/conda/en/4.6.0/\\_downloads/52a95608c49671267e40c689e0bc00ca/conda-cheatsheet.pdf](https://docs.conda.io/projects/conda/en/4.6.0/_downloads/52a95608c49671267e40c689e0bc00ca/conda-cheatsheet.pdf). 2
- [3] “Conda Installation.” <https://conda.io/projects/conda/en/latest/user-guide/install/index.html>. 2
- [4] “GNU Image Manipulation Program.” <https://docs.gimp.org/2.10/en/>. 2
- [5] “IrfanView.” <https://www.irfanview.com/>. 2
- [6] “ImageMagick - Command-line Tools.” <https://imagemagick.org/script/command-line-tools.php>. 2
- [7] “Displaying the coordinates of the points clicked on the image using Python-OpenCV.” <https://www.geeksforgeeks.org/displaying-the-coordinates-of-the-points-clicked-on-the-image-using-python-opencv/>. 3